

EXAMEN

Nombres : Milagros Guadalupe

Apellidos : Siancas Ramirez

Código : 22200154

Fecha : 20/06/2026

Fase I:

- 1) El SSOT Se considera la Unica fuente porque centraliza el acceso a los datos, ningun otro componente lee la BD o la API directamente, todos pasan por él. Es una fachada porque oculta la complejidad de Coordinar multiples fuentes detras de una interfaz simple. En la coordinación el repositorio expone un Flow que lee ROOM, cuando necesita data fresca, llama a Retrofit, guarda la respuesta en ROOM, y ROOM emite automáticamente el cambio, la UI solo observa el Flow del Repositorio, sin saber si vino de red o de BD.
- 2) El ViewModelScope es un CoroutineScope atado al ciclo de vida del ViewModel, que se cancela automáticamente en onCleared(), evitando el memory leaks por coroutines huérfanas. Si el usuario cierra la pantalla antes de la respuesta, el scope se cancela junto con la coroutine, liberando recursos sin necesidad de cancelación manual.
- 3) Flow es Frio, no ejecuta nada hasta que hay un collector y se reejecuta por cada nuevo suscriptor. StateFlow es caliente y existe independencia de los collectors, mantiene siempre un valor actual (.value) y lo comparte entre todos. Se prefiere StateFlow para UI porque sobrevive a cambios de configuración, evita recalcular datos por cada observador, y siempre tiene un valor disponible para renderizar.
- 4) DI Manual con Application, la clase Application instancia manualmente la BD y el repositorio; el ViewModel accede a ellos via application o MainActivity, garantizando una sola instancia compartida en toda la app. By lazy retrasa la creación del objeto hasta su primer uso, asegurando que se cree una sola vez y de forma thread-safe, evitando trabajo innecesario al iniciar la app.

FASE 2

- 1 a) El Flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow
- 2 b) El operador Dispatchers.Default permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas CPU como el procesamiento masivo de datos
- 3 c) Para que un servicio en primer plano de ubicación sea válido en Android 10, debe declararse en el manifiesto con el tipo android:foregroundServiceType="location"
- 4 d) La función getInstances() de la clase Application es la que garantiza que los componentes compartidos compartan la misma instancia del repositorio mediante un "cast"
- 5 e) El componente que permite transformar una API basada en callbacks antiguo en una función suspendible moderna se llama suspendCancellable Coroutine
- 6 f) El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml
- 7 g) En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denominan HAL (Hardware Abstraction Layer)
- 8 h) El operador de Flow que permite combinar múltiples flujos (como GPS y Sensores) para emitir un estado unificado es combine

FASE III 1) Repositorio y DI

```
// DAO de referencia
interface UsuarioDao {
    suspend fun ObtenerUsuarios(): List<Usuario>
}
```

```
// Repositorio que recibe el DAO por constructor
class UsuarioRepository (private val usuarioDao: UsuarioDao) {
    suspend fun obtener Usuarios(): List<Usuario> {
        return usuarioDao.obtenerUsuarios()
    }
}
```

```
// Instanciación en la clase Application usando by lazy
class MyApp : Application() {
    private val database by lazy { AppDatabase.getInstance(this) }
```

```
    val usuarioRepository by lazy {
        UsuarioRepository(database.usuarioDao())
    }
}
```

2) Lógica dinámica de Identidad

```
a) data class Usuario (
    val nombre: String,
    val apellidos: String,
    val edad: Int?
)
```

```
fun main() {
```

```
    // b) Objeto con datos reales
```

```
    val usuario = Usuario (
        nombre = "Milagros",
        apellidos = "Siancas Ramirez",
        edad = 24
    )
```

```
    // c) Cálculo del Puntaje de Identidad
```

```
    val puntajeCalculado: Int? = usuario.nombre.length + usuario.apellidos.length
```

```
    // e) Operador Elvis
```

```
    val puntajeIdentidad = puntajeCalculado ?: 0
```

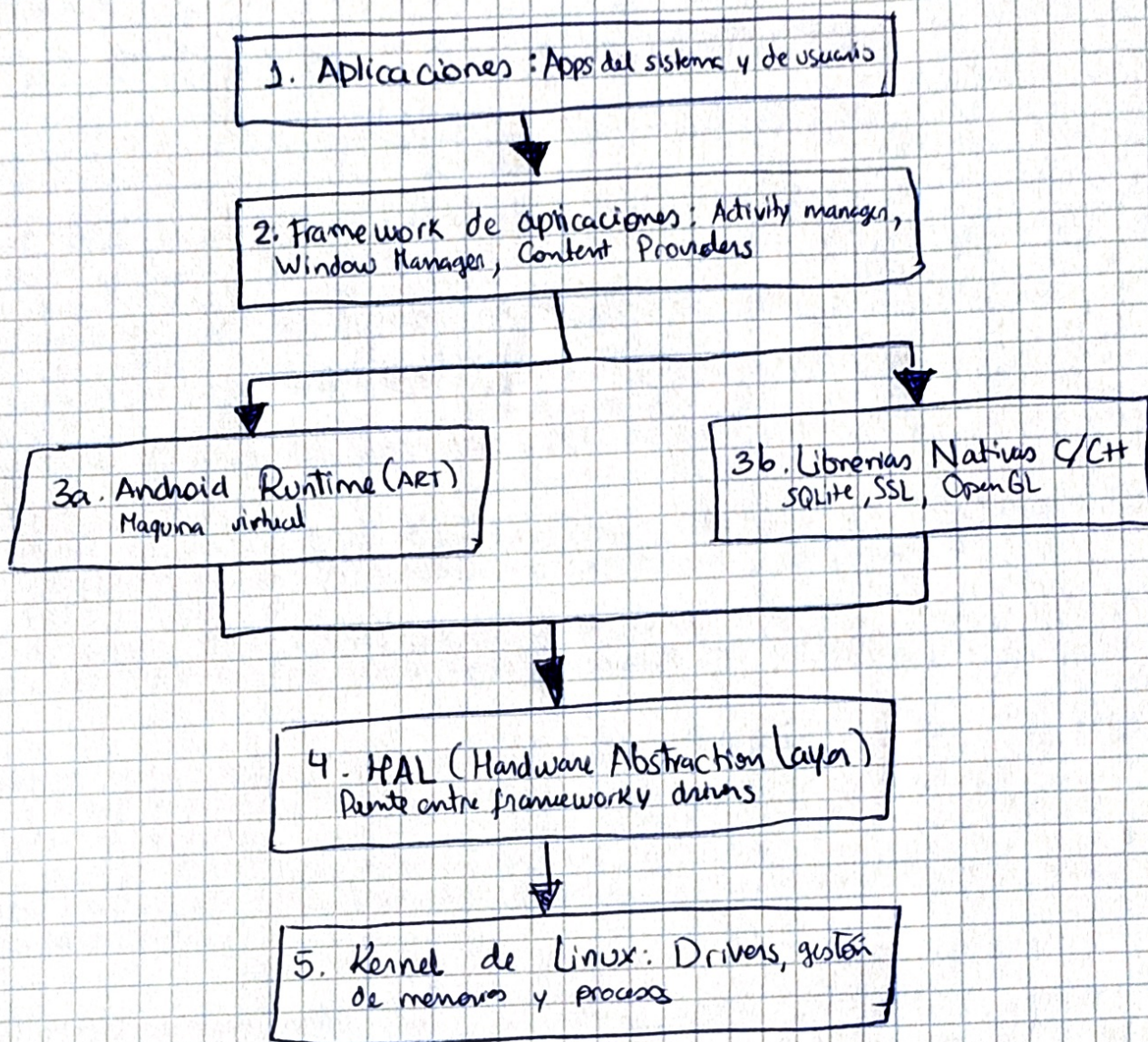
```
    // d) Impresión del mensaje
```

```
    val nombreCompleto = "${usuario.nombre} ${usuario.apellidos}"
    println ("Usuario: $nombreCompleto, tu puntaje de identidad es: $puntajeIdentidad")
```

Salida esperada

Usuario: Milagros Siancas Ramirez, tu puntaje de identidad es: 23

FASE 4



- 1. Aplicaciones:** Capa superior donde residen las apps que usa el usuario final, única capa donde interactúa directamente con el usuario.
- 2. Framework de aplicaciones:** Conjunto de APIs que los desarrolladores usamos para construir apps. Expone los servicios del sistema de forma estandarizada.
- 3. Android Runtime y Librerías nativas:** el primero es el entorno donde se ejecuta el código compilado en la máquina virtual que contiene en las apps, el segundo son bibliotecas de bajo nivel compartidas por el sistema.
- 4. HAL:** Capa que actúa como puente estandar entre el Framework y los drivers del Kernel, define interfaces genéricas.
- 5. Kernel de Linux:** Capa + baja, gestiona los drivers de hardware, la memoria, los procesos y la comunicación entre ellos. Es donde se apoya todo el sist. operativo.